

IM05 – Challenge Report

Héloïse Gouarné

0. Sommaire

1 – Approche du problème	Page 1
2 – Prétraitements	Page 2
3 – Modèle	Page 4
4 – Processus d’entraînement du modèle	Page 5
5 – Acquisition des résultats	Page 6
6 – Analyse des résultats	Page 7

1. Approche du problème

Pour me familiariser avec les données, j’ai commencé par une approche simple. J’ai créé un modèle de **Random Forest** qui m’a permis d’avoir un score d’à peu près 20%. Je savais que j’allais m’orienter vers un modèle de **Convolutionnal Neural Networks**. J’ai d’abord essayé de faire mes propres modèles à la main, mais j’ai fini par importer des outils plus profonds et pré entraînés vus en classe. Pour adapter mon modèle à la compétition, j’ai remplacé les dernières couches par d’autres, choisies pour adapter l’architecture importée au problème à résoudre.

Dans mon notebook, vous pourrez voir plusieurs outputs : les attributions des classes finales (submission), mais aussi l’encodeur et le modèle lui-même. C’est une mesure de sécurité que j’ai prise car mon code s’arrêtait parfois avant la fin, faute de GPU ou d’erreurs difficiles à détecter (car elles survenaient toujours après plus d’une demi-heure d’entraînement).

Ceci m’a permis de tester les modèles sur leurs encodeurs spécifiques (sinon il m’arrivait d’obtenir des scores très bas à cause d’une mauvaise correspondance label-encodeur), et de tester ma manière de prédire les labels et ma stratégie de post-traitement sans devoir relancer l’intégralité de mon entraînement.

Finalement, la globalité de mon code utilise la bibliothèque PyTorch, par soucis de cohérence, ainsi que pour les avantages qu’elle propose. L’utilisation de Loaders et sa flexibilité ont notamment aidé. C’est de plus la bibliothèque utilisée en cours et travaux pratiques, donc ce choix me semble cohérent comme continuité des apprentissages de l’UE.

Note des versions utilisées :

PyTorch version: 2.10.0+cu128

OpenCV version: 4.13.0

Numpy version: 2.0.2

Pandas version: 2.3.3

Pillow (PIL) version: 11.3.0

Scikit-learn version: 1.6.1

2. Prétraitements

Je me suis largement appuyée sur les articles fournis pour choisir les pré-traitements adaptés.

Commençons par observer les transformations que je fournis aux loaders :

```
transforms.Resize((288, 288)),
transforms.RandomHorizontalFlip(),
transforms.RandomVerticalFlip(),
transforms.RandomRotation(90),
transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9,
1.1)),
transforms.ColorJitter(brightness=0.1, contrast=0.1,
saturation=0.1),
transforms.ToTensor(),
transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
```

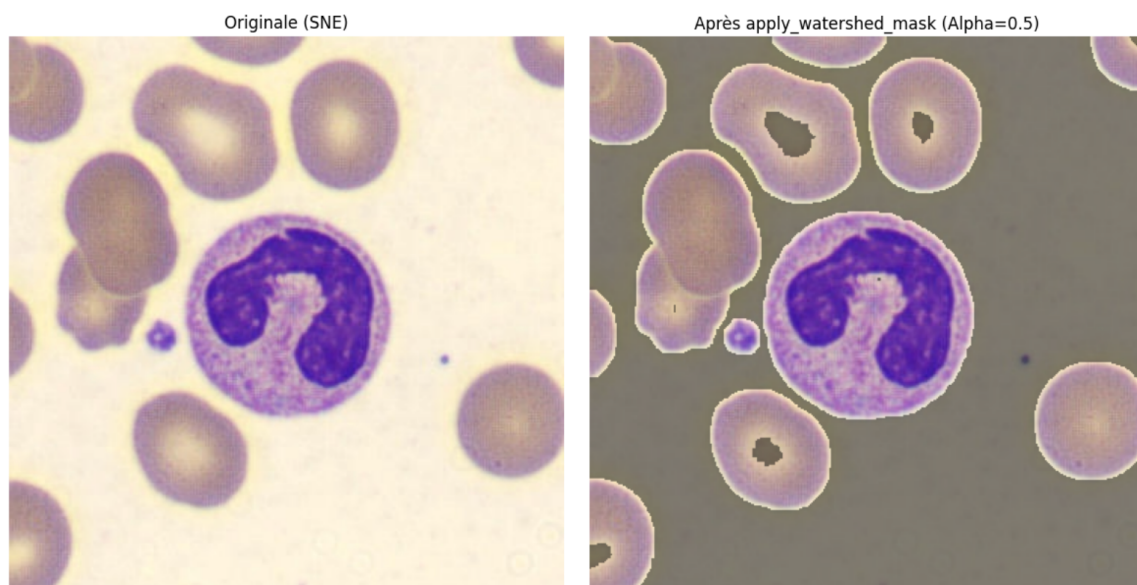
- Le premier « Resize » uniformise la taille des images en entrées du CNN. La résolution à 288 est assez haute, pour essayer de capter le plus de détails possibles. J'ai essayé de la mettre encore plus haute, mais l'exécution prenait plus de 15 heures, avant d'être annulée. 288 est donc une valeur qui offre un compromis entre réalisabilité et précision.
- Les flips retournent l'image, car on peut voir la cellule sous n'importe quel angle dans le microscope. C'est de même pour la rotation. On apprend ainsi mieux la forme globale, et la prédiction ne dépend pas d'une orientation particulière.
- La partie affine zoom/dézoom et translate légèrement les images, pour la même raison, que les flips et la rotation. Tout ceci constitue aussi de la **Data Augmentation** : on ajoute des images construites avec les images du Dataset d'origine pour mieux entraîner le modèle, et sur une plus grande base.
- Le ColorJitter fait aussi ceci, en modifiant la luminosité, le contraste et la saturation des images. Ainsi si les cellules ont été plus ou moins trempées dans le colorant du laboratoire, ou si les images sont plus ou moins lumineuses que l'image moyenne du dataset d'entraînement, le modèle ne sera pas trop perturbé.
- Les deux dernières transformations s'assurent que les images soient au bon format pour être mises en entrée du ConNeXt importé.

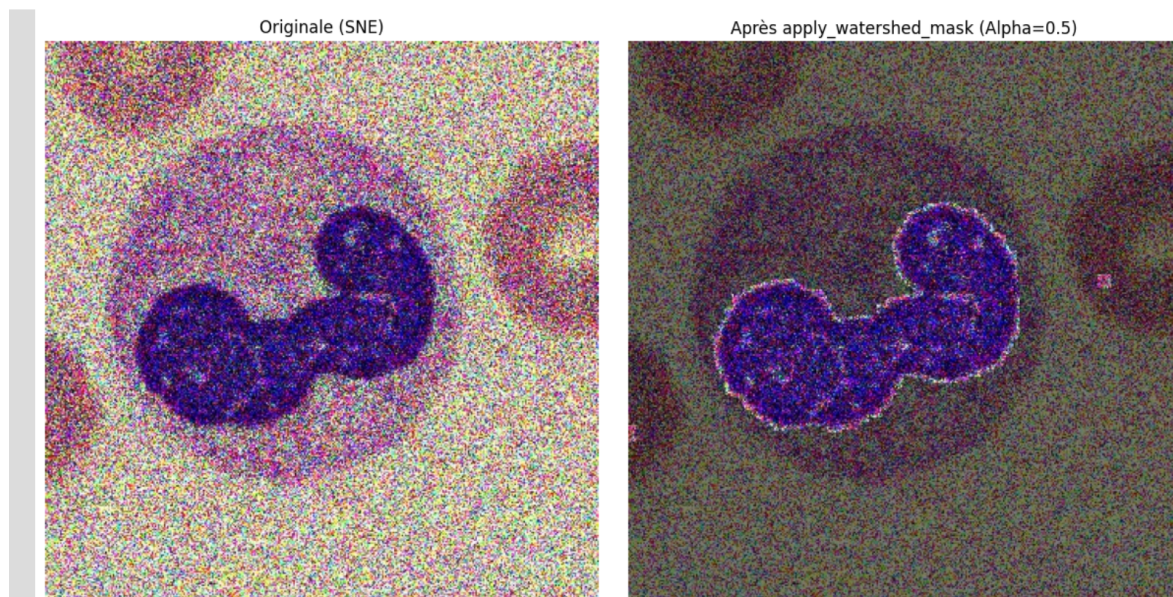
J'ai appliqué une **normalisation de l'intensité** dans mes loaders, avec l'outil « `ImageOps.autocontrast(image)` » de la bibliothèque PIL. L'idée derrière était d'uniformiser les images vues par le modèle. En effet, les photos provenant des microscopes peuvent varier en contraste, qualité, luminosité selon qui les a prises, où, comment... Ce pré-traitement est donc un des premiers que j'ai appliqués, pour permettre de remettre toutes les images du Dataset fournies sur un pied d'égalité.

L'outil **CLAHE** (Contrast Limited Adaptive Histogram Equalization), de la bibliothèque cv2, est utilisé pour faire ressortir le contraste et récupérer les détails possiblement perdus dans la normalisation. Comme cet outil a un effet local, et la normalisation d'intensité est globale, ces deux méthodes se combinent bien pour obtenir des images lisibles mais détaillées. Les détails des cellules et leurs textures sont en effet un élément qui peut permettre de les distinguer, donc j'ai choisi ce pré-traitement pour affiner cette analyse.

De plus, j'applique un **watershed** à mes images. C'est une méthode proposée dans un des articles, qui a pour but d'aider le modèle à repérer la cellule. J'ai d'abord appliqué le watershed pour ne garder que la cellule, coupant le reste de l'image. Ceci était trop brutal, car la fonction de partage des eaux n'est pas un outil parfait, et pouvait segmenter de manière erronée. De plus, le contexte sanguin peut être utile à la classification. Au final, j'ai donc opté pour un « mask » : j'applique le watershed et assombrit les parties de l'image qui ne sont sûrement pas dans la cellule, ce qui m'évite de courir le risque de perdre trop d'information. Le masque d'origine étant trop brutal (un coefficient de 0.3) j'ai lui-même dû le tester en termes d'intensité, pour déterminer un bon compromis entre assombrissement et perte d'information. J'ai abouti à un coefficient de 0.5 : chaque pixel hors de la cellule voit son intensité divisée par deux.

La fonction watershed n'en est pas vraiment une, car la méthode que j'avais d'abord essayée (cv2) prenait trop longtemps, et n'était pas extrêmement efficace. J'ai donc choisi de séparer les classes avec la **méthode d'Otsu**, plus robuste, mais j'ai gardé le nom watershed car c'est l'article qui m'en a donné l'idée, et que je l'utilise de la même manière.





Comme vous pourrez le voir ci-dessus, cette fonction isole l'ensemble des structures biologiques présentes. Cela permet de voir le contexte sanguin de la cellule observée, tout en la faisant se détacher du fond de l'image (pour la première cellule, elle est maintenant bien plus repérable). Le masque à 0.5 est crucial pour ne pas donner trop d'importance au contexte sanguin, qui est séparé du fond mais n'est pas non plus la seule autre feature que la cellule (petit point dans le fond de l'image...). L'image d'en bas montre une autre raison d'appliquer ce masque à 0.5 : le seuillage d'Otsu peut ne pas séparer que la cellule, ici il ne garde que le noyau. De toute manière, cette image est difficile à interpréter, à cause d'un bruit très fort et d'une indépendance des pixels voisins. Mais ici, le modèle saura au moins bien repérer le noyau du globule blanc.

J'ai hésité à appliquer le filtre de Wiener, une autre solution proposée par les articles. Cependant, celui-ci multipliait mon temps d'exécution par trois, et son effet de débruitage adapté est rattrapé par la combinaison watershed-normalisation-CLAHE, d'où son omission du code final.

3. Modèle

La base de mon modèle est l'importation de **ConvNeXt**. Ce modèle est un des plus efficace, donc je me suis appuyée sur sa structure, en l'adaptant au problème. De plus, c'est un modèle adapté au motifs locaux, et aux détails, ce qui est adapté à la classification de ces globules blancs. Il utilise de plus des larges noyaux de convolutions, et qui saura donc comparer la taille du noyau à la taille de la cellule dans sa globalité, ce qui est un autre avantage pour la classification des globules blancs.

J'ai remplacé la tête du modèle ConvNeXt, comme ci-dessous :

```
model.classifier[2] = nn.Sequential(
    nn.Linear(in_features, 512),
    nn.BatchNorm1d(512),
```

```
nn.ReLU(),
nn.Dropout(0.4),
nn.Linear(512, nb_cl)
```

La couche d'origine est une simple projection dans un espace plus petit. Ici, je projette d'abord dans un espace de 512 neurones, afin de faire une première combinaison des nombreuses features en entrée de la dernière couche. Ceci permet d'associer les features qui permettent d'aboutir à la même classification. Le BatchNorm stabilise ce résultat, avant de l'envoyer en entrée à un ReLU, qui induit la complexité du modèle non linéaire. Le Dropout juste après permet d'éviter l'overfitting, et de mieux entraîner l'intégralité des neurones, puisqu'on en éteindra 40% au hasard à chaque étape, ce qui force les 60% restant à s'adapter et à savoir repérer le plus d'indices de classification possible. Le dernier Linear permet d'effectuer la classification finale, et le modèle renvoie donc la probabilité associée à chaque classe, pour l'image d'entrée.

Le modèle possède un argument `freeze_backbone`, que j'active dans sa version finale. Ceci permet de ne pas entraîner le corps du modèle, importé, mais seulement la tête détaillée ci-dessus, pour les premières étapes. Il s'agit de la seule partie du modèle que j'ai intégralement créée, et elle est donc totalement « ignorante », contrairement à ConvNeXt qui est pré entraîné. Le fait de l'entraîner seule au début permet de la remettre à niveau, et d'éviter que son ignorance fasse perdre des données repérées par le corps du modèle, plus compétent. Après trois étapes, l'ensemble du modèle sera traité de la même manière, et entraîné de manière « standard ». Le code procède à une mise à jour de `train_loader`, ce qui correspond à un ajustement de la learning rate (le pas), qui doit être plus haute pour les premières étapes, vu qu'il faut vite mettre à niveau des couches auparavant totalement ignorantes, aux poids aléatoires.

4. Processus d'entraînement du modèle

Cette section a pour but d'expliquer l'intégralité des décisions prises lors du codage de la boucle d'entraînement.

Avant la boucle, les paramètres d'apprentissages sont définis :

```
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
model = get_convnext_model(num_classes=13,
freeze_backbone=True).to(device)
optimizer = torch.optim.AdamW(model.classifier.parameters(), lr=1e-3,
weight_decay=0.01) # nb: optimizer pr la tête
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode='max', factor=0.2, patience=3)

patience = 10
epochs = 40
unfrozen = False
```

Le critère **CrossEntropyLoss** est utilisé car il s'agit de la fonction de perte la plus adaptée à un problème multi classe, comme c'est le cas ici. Le `label_smoothing` mis en paramètre précise au modèle qu'il pense qu'une cellule appartient à une classe donnée

A à seulement 90% (contre une certitude de 100% standard). Ceci permet de créer un modèle plus général, qui se fiera sûrement moins aux images qui ne l'aide pas. Par exemple, si une image a le label A mais elle ressemble physiquement beaucoup à une autre classe B, le modèle ne commencera pas à confondre ces deux classes.

L'optimizer est essentiellement l'optimizer Adam, qui présente des avantages de momentum et de taux d'apprentissage adapté. **AdamW** en est une version plus adaptée au système de `weight_decay`, que l'on voit dans ses paramètres. Il s'agit de pénalité sur les trop gros poids, et prévient ainsi l'overfitting, et le problème du gradient qui explose. Pour un modèle de classification dont certaines classes sont très peu représentées, comme ici, cela m'a paru être un bon outil pour éviter que les classes les plus communes pèsent trop dans les décisions, et écrasent les autres.

Le **scheduler** permet de diviser le learning rate (pas) si le modèle n'apprend plus. Il surveille le score F1 et agit s'il n'a pas bougé après trois itérations. Ceci permet au modèle de s'adapter lui-même à son étape d'apprentissage.

La patience correspond à une patience d'**Early Stopping** : si le modèle n'apprend plus depuis 10 itérations, il s'arrête. Ce n'est en pratique pas le cas, surtout depuis que j'applique le scheduler. Il s'agit ici d'une sécurité, et il a été utilisé pour des versions précédentes de mon code.

Les 40 itérations sont plus que la moyenne recommandée pour ConNeXt (15), mais au vu des changements qui ont lieu lors de l'entraînement (le pas qui change, le corps qui commence à apprendre à l'étape 3...), il s'agit d'une précaution. La logique d'Early Stopping couperait l'apprentissage s'il devenait obsolète, donc j'ai choisi un nombre plutôt haut. Ici, le modèle s'entraînera pendant à peu près 7 heures.

Le bloc suivant s'applique à la condition « `if epoch >= 3 and not unfrozen:` ». Il a pour but de baisser le pas d'apprentissage et d'adapter les paramètres décrits ci-dessus lorsque le corps du modèle commence à apprendre. Celui-ci étant un modèle pré-entraîné, on veut l'adapter aux images d'entrées, pas le changer entièrement. Il a donc un pas relativement bas (10^{-5}). La tête garde un pas dix fois plus grand, car il s'agit toujours d'un maillon plus faible que le reste, qui va donc être mené à plus bouger.

Le reste de la boucle d'entraînement est assez classique. Je calcule à chaque itération les pertes et le score F1 selon le set de validation, initialisé au début du code. Il permet de sauvegarder la version du modèle la plus intéressante et pas forcément la dernière (s'il y a eu une sur correction par exemple). Sinon, cela m'a permis d'avoir une visibilité sur l'entraînement lors de l'exécution.

5. Acquisition des résultats

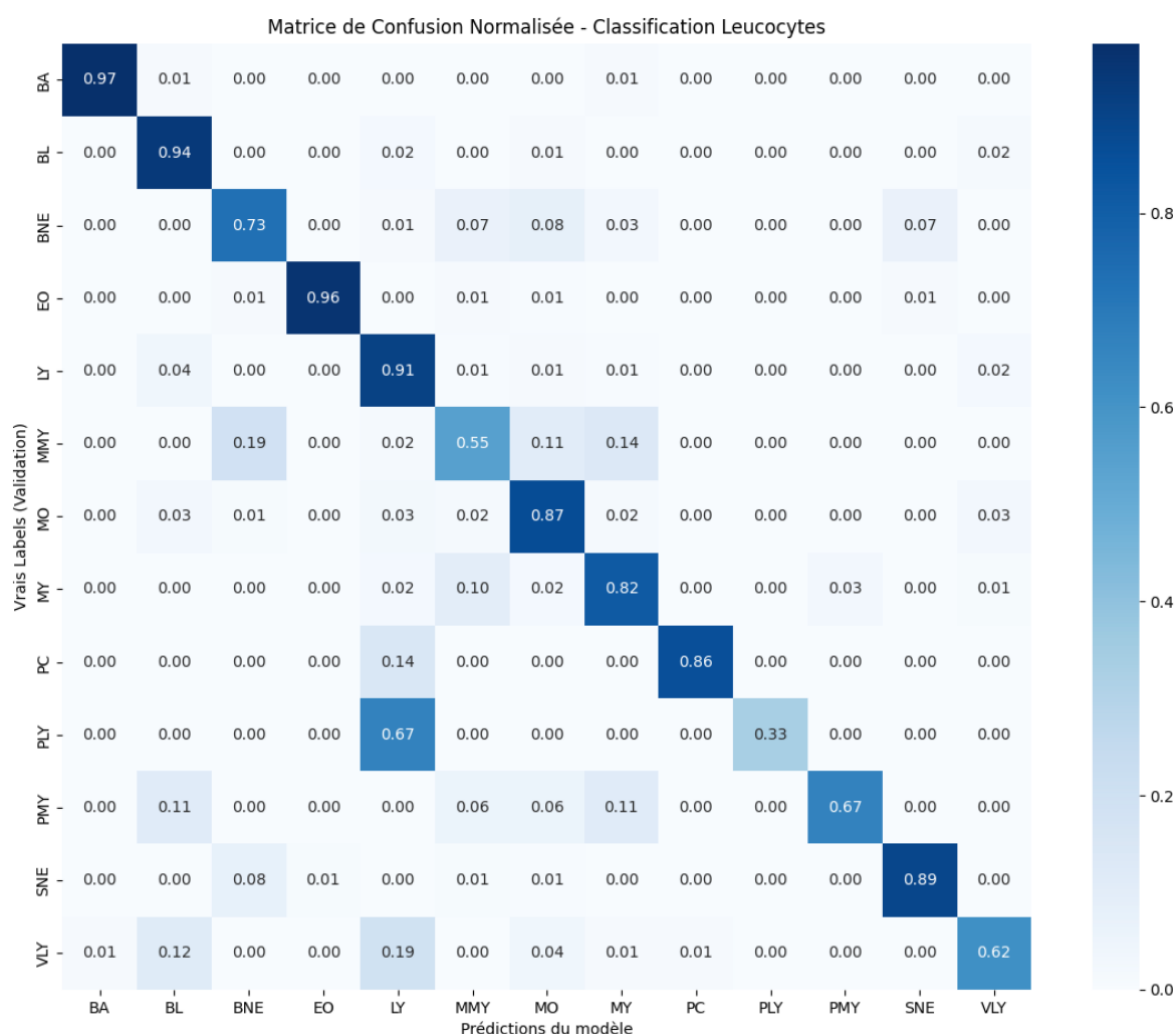
Les résultats sont obtenus par l'utilisation d'une fonction « `get_predictions` ». Celle-ci utilise une technique de **Test Time Augmentation** (TTA), pour obtenir une meilleure fiabilité sur le résultat. La prédiction se fait en utilisant la fonction softmax, préconisée pour les classifications multi-classes.

Concrètement, la fonction prédit, pour chaque image, la probabilité de chaque classe pour l'image telle qu'elle (via le `test_loader`), puis pour l'image inversée horizontalement, puis verticalement, puis après avoir subi une rotation de 90 degrés.

La prédiction réelle est faite sur la moyenne des quatre distributions de probabilités. Elle la calcule et prend la classe la plus probable selon elle (argmax). Ceci permet d'éviter d'avoir un biais de position (par exemple si tous les lymphocytes du TrainDataset étaient à droite de leurs images respectives, une cellule à droite ne serait pas automatiquement labellee comme lymphocyte). Il s'agit d'une analyse plus fine que l'application simple de la fonction softmax à la sortie du modèle.

6. Analyse des résultats

Voici une matrice de confusion, réalisée grâce au set de validation utilisé lors de l'entraînement :



Globalement, quelle que soit la classe, sauf PLY, elle est correctement attribuée plus de la moitié des cas, et la matrice est clairement presque diagonale : le modèle fonctionne bien sur 12 des 13 classes.

Le plus gros problème est la confusion entre les classes LY et PLY. LY est plus commune et a des caractéristiques communes à PLY, d'où la confusion. Pour mieux entraîner le modèle, il faudrait pénaliser le modèle dès qu'on voit qu'il a commis cette erreur, pendant la partie

validation de la boucle d'entraînement. J'avais tenté de mettre un système similaire en place sur l'ensemble des classes rares, qui n'avait pas abouti car je n'avais pas réussi à trouver un juste milieu, où le modèle était pénalisé sans pour autant qu'il oublie son entraînement, et j'obtenais donc des résultats en dessous des précédents, pour un entraînement très longs. Cependant, la détection de la classe PLY n'est clairement pas satisfaisante dans la version finale du modèle. Le reste des classes sont repérées au moins la moitié du temps, et les classes communes sont très bien classées, ce qui demeure une base solide.